Modulo 4

Strutture dati e algoritmi: un approccio pratico

Strutture dati e algoritmi: un approccio pratico

- Strutture dati e algoritmi: concetti fondamentali di strutture dati e algoritmi: liste, tuple, dizionari, set, stack, queue, linkedlists, algoritmi di ricerca e algoritmi di ordinamento.
- Analisi della complessità: concetto di complessità temporale e spaziale, e mostra come utilizzare la notazione Big O per analizzare la complessità di un algoritmo.
- Strutture dati avanzate: strutture dati avanzate, come alberi e grafi.

Strutture dati e algoritmi: un approccio pratico

- Liste: proprietà, operazioni e metodi comuni
- Tuple: definizione e quando usarle
- Dizionari: chiavi, valori e metodi
- Set: definizione, operazioni e casi d'uso
- Stacke Queue: definizione e implementazione
- LinkedLists: introduzione e implementazione
- Algoritmi di ricerca: ricerca lineare e binaria
- Algoritmi di ordinamento: bubblesort, insertionsort e altri
- Analisi della complessità temporale e spaziale (Big O notation)
- Strutture dati avanzate e utilizzo in Python
- Alberi e grafi: introduzione e concetti base

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Liste: proprietà, operazioni e metodi comuni

Liste

Le liste in Python sono strutture dati flessibili e potenti, utilizzate per memorizzare collezioni di elementi. Sono simili agli array in altri linguaggi di programmazione, ma offrono una maggiore flessibilità.

```
# Creazione di una lista
frutti = ["mela", "banana", "ciliegia"]
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Liste: proprietà, operazioni e metodi comuni

Proprietà delle Liste

1. **Dinamiche:** La dimensione di una lista può cambiare, ovvero può crescere o diminuire al bisogno.
2. **Ordinate:** Gli elementi in una lista mantengono l'ordine in cui sono stati aggiunti.
3. **Accessibili tramite Indice:** Gli elementi possono essere accessibili tramite un indice. Gli indici iniziano da 0.
4. **Eterogenee:** Una lista può contenere elementi di diversi tipi, inclusi altri oggetti complessi come altre liste.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Liste: proprietà, operazioni e metodi comuni

Operazioni Comuni sulle Liste

- **Aggiungere Elementi:**
  - **append(elemento)**: aggiunge un elemento alla fine della lista.
  - **insert(indice, elemento)**: inserisce un elemento in una posizione specifica.
- **Rimuovere Elementi:**
  - **remove(elemento)**: rimuove la prima occorrenza dell'elemento specificato.
  - **pop([indice])**: rimuove e restituisce un elemento in una posizione specifica.
- **Concatenazione:** **+** può essere utilizzato per concatenare due liste.
- **Replicazione:** ***** può essere utilizzato per ripetere gli elementi di una lista per un dato numero di volte.
- **Accesso agli Elementi:** Utilizzare l'indicizzazione **lista[indice]** e lo slicing **lista[inizio:fine]**.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Liste: proprietà, operazioni e metodi comuni

Metodi Comuni

Le tuple in Python sono strutture dati simili alle liste, ma con alcune differenze chiave. Sono utilizzate per memorizzare collezioni di elementi, ma a differenza delle liste, le tuple sono immutabili, il che significa che una volta create, i loro elementi non possono essere modificati.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Liste: proprietà, operazioni e metodi comuni

Esempio di codice

```
# Creazione di una lista (riutilizziamo l'esempiodella
prima slide del modulo 4)
frutti = ["mela", "banana", "ciliegia"]

# Aggiunta di un elemento
frutti.append("arancia")

# Accesso agli elementi
print(frutti[1])  # stampa "banana"

# Slicing
print(frutti[1:3])  # stampa ['banana', 'ciliegia']

# Ordinamento della lista
frutti.sort()
print(frutti)  # ['arancia', 'banana', 'ciliegia', 'mela']

# Inversione dell'ordine
frutti.reverse()
print(frutti)  # ['mela', 'ciliegia', 'banana', 'arancia']
```

Le liste sono tra le strutture dati più utilizzate in Python e sono incredibilmente utili per una vasta gamma di compiti di programmazione, dalla manipolazione dei dati all'algoritmo di base e ai compiti di programmazione avanzata.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Tuple: definizione e quando usarle

Tuple

- **len(lista)**: restituisce il numero di elementi nella lista.
- **sort()**: ordina gli elementi della lista sul posto.
- **reverse()**: inverte l'ordine degli elementi nella lista.
- **index(elemento)**: restituisce l'indice del primo elemento con il valore specificato.
- **count(elemento)**: conta quante volte un elemento appare nella lista.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Tuple: definizione e quando usarle

Definizione e utilizzo

- **Immutabili:** Una volta creata, una tupla non può essere modificata.
Non è possibile aggiungere, rimuovere o modificare gli elementi.
- **Ordinate:** Come le liste, le tuple mantengono l'ordine degli elementi.
- **Accessibili tramite Indice:** Gli elementi possono essere accessibili tramite indici, partendo da 0.
- **Eterogenee:** Possono contenere elementi di diversi tipi.

Le tuple sono definite racchiudendo gli elementi tra parentesi tonde **()**.

```
mia_tupla = (1, "mela", 3.14)
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Tuple: definizione e quando usarle

Operazioni Comuni

- **Accesso agli Elementi:** Utilizzando l'indicizzazione **tupla[indice]** e lo slicing **tupla[inizio:fine]**.
- **Concatenazione: +** può essere utilizzato per unire tuple.
- **Replicazione: *** può essere utilizzato per ripetere gli elementi di una tupla.
- **Unpacking:** Assegnare gli elementi di una tupla a variabili separate.
- **len(tupla)**: Restituisce la lunghezza della tupla.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Tuple: definizione e quando usarle

Esempio di codice

```
# Creazione di una tupla
mia_tupla = ("mela", "banana", "ciliegia")

# Accesso agli elementi
print(mia_tupla[1])  # stampa "banana"

# Slicing
print(mia_tupla[1:3])  # stampa ('banana',
'ciliegia')

# Unpacking
(frutto1, frutto2, frutto3) = mia_tupla
print(frutto1)  # stampa "mela"
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Tuple: definizione e quando usarle

Quando Usare le Tuple

•**Efficienza:** Le tuple, essendo immutabili, sono generalmente più leggere e richiedono meno memoria rispetto alle liste. Sono quindi più efficienti dal punto di vista delle prestazioni per collezioni di dati statici.
•**Sicurezza del Codice:** Se hai bisogno di una sequenza di elementi che non deve essere modificata, le tuple sono una scelta migliore per evitare modifiche accidentali.
•**Chiave del Dizionario:** Le tuple possono essere utilizzate come chiavi in un dizionario, a differenza delle liste.
•**Ritorno di Più Valori da una Funzione:** Le tuple sono comunemente usate per restituire più valori da una funzione.

Le tuple, quindi, sono utili quando si necessita di una lista immutabile di elementi, per ragioni di efficienza o di design del codice.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Dizionari: chiavi, valori e metodi

Dizionari

I dizionari in Python sono strutture dati flessibili e potenti, usate per memorizzare e gestire collezioni di elementi in coppie chiave-valore. Sono dinamici, possono crescere o diminuire al bisogno, e sono una componente fondamentale della programmazione in Python.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Dizionari: chiavi, valori e metodi

Caratteristiche dei Dizionari

- **Chiave-Valore:** Ogni elemento in un dizionario è una coppia chiave-valore. Le chiavi fungono da identificatori unici.
- **Mutabili:** I dizionari possono essere modificati. È possibile aggiungere, rimuovere e modificare i valori.
- **Ordine di Inserimento:** A partire da Python 3.7, i dizionari mantengono l'ordine di inserimento degli elementi.
- **Chiavi Uniche:** Ogni chiave deve essere unica nel dizionario.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Dizionari: chiavi, valori e metodi

Operazioni Comuni

- **Aggiungere Elementi: dizionario[chiave] = valore**.
- **Rimuovere Elementi: del dizionario[chiave]** o **dizionario.pop(chiave)**.
- **Accesso ai Valori: dizionario[chiave]**.
- **Verifica dell'Esistenza di una Chiave: chiave in dizionario**.
- **Iterazione:** Tramite metodi come **dizionario.keys()**, **dizionario.values()**, e **dizionario.items()**.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Dizionari: chiavi, valori e metodi

Set: definizione, operazioni e casi d’uso

I set in Python sono collezioni non ordinate di elementi unici. Sono simili alle liste e ai dizionari, ma con alcune differenze chiave che li rendono utili per specifici casi d'uso.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Dizionari: chiavi, valori e metodi

Definizione dei Set

- **Non Ordinati:** Gli elementi in un set non mantengono alcun ordine specifico.
- **Valori Unici:** Ogni elemento in un set è unico; non sono ammessi duplicati.
- **Mutabili:** È possibile aggiungere o rimuovere elementi da un set, ma gli elementi stessi contenuti nel set devono essere immutabili (come numeri, stringhe e tuple).
- **Sintassi:** I set sono definiti racchiudendo gli elementi tra parentesi graffe **{}**, o utilizzando il costruttore **set()**.

```
mio_set = {1, 2, 3}
altro_set = set([4, 5, 6])
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Dizionari: chiavi, valori e metodi

Operazioni Comuni

- **Aggiunta e Rimozione: mio_set.add(elemento)** per aggiungere, **mio_set.remove(elemento)** per rimuovere.
- **Unione: set1 | set2** o **set1.union(set2)**.
- **Intersezione: set1 & set2** o **set1.intersection(set2)**.
- **Differenza: set1 - set2** o **set1.difference(set2)**.
- **Differenza Simmetrica: set1 ^ set2** o **set1.symmetric_difference(set2)**.
- **Controllo Appartenenza: elemento in mio_set**.
- **Iterazione:** Simile alle liste e ai dizionari.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Dizionari: chiavi, valori e metodi

Esempio di Codice

```
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}

# Unione
print(set1 | set2)  # {1, 2, 3, 4, 5, 6}

# Intersezione
print(set1 & set2)  # {3, 4}

# Differenza
print(set1 - set2)  # {1, 2}

# Differenza Simmetrica
print(set1 ^ set2)  # {1, 2, 5, 6}
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Dizionari: chiavi, valori e metodi

Casi d'Uso dei Set

1. **Eliminazione dei Duplicati:** Convertire una lista in un set per rimuovere automaticamente tutti i duplicati.
2. **Operazioni di Appartenenza:** I set sono ottimizzati per le operazioni di appartenenza (ad esempio, controllo se un elemento è presente nel set), che sono più veloci rispetto alle liste e ai dizionari.
3. **Operazioni di Insieme:** I set facilitano le operazioni matematiche di insieme, come unione, intersezione, e differenza.
4. **Filtraggio Dati:** Utilizzare i set per filtrare dati da altre collezioni.

I set sono particolarmente utili quando è necessario mantenere una collezione di elementi unici e quando le operazioni di insieme sono frequentemente richieste. La loro unicità e le efficienti operazioni di appartenenza li rendono uno strumento potente in diversi scenari di programmazione.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Stack e Queue: definizione e implementazione

Stack e Queue

In Python, stack e queue sono due tipi di strutture dati che gestiscono gli elementi in modi specifici, ognuna con le proprie regole per l'aggiunta e la rimozione di elementi.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Stack e Queue: definizione e implementazione

Stack (Pila)

Uno stack è una struttura dati di tipo LIFO (Last In, First Out), dove l'ultimo elemento aggiunto è il primo ad essere rimosso.
Si tratta più di un concetto di informatica che di una precisa struttura predisposta in Python.
Come detto, nella programmazione, uno stack è un concetto astratto che definisce una modalità specifica di gestire i dati: LIFO (Last In, First Out), ovvero l'ultimo elemento inserito è il primo a essere rimosso.
In Python si possono implementare, essenzialmente, con le liste.
In Python, quando scrivi **stack = []**, stai effettivamente creando una lista. Tuttavia, puoi utilizzare una lista in Python come uno stack grazie al modo in cui puoi aggiungere e rimuovere elementi da essa.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Stack e Queue: definizione e implementazione

Stack :: Come una Lista Agisce da Stack

Per usare una lista come uno stack in Python, si utilizzano due operazioni principali:

- **Push (aggiungere/append):** Usa **append()** per aggiungere un elemento in cima allo stack (alla fine della lista).
- **Pop (rimuovere/pop):** Usa **pop()** senza indicizzare per rimuovere l'elemento in cima allo stack (l'ultimo elemento della lista).

Ecco un esempio:

```
stack = []          # Crea una lista vuota

stack.append(1)     # Push: aggiunge 1 in cima allo stack
stack.append(2)     # Push: aggiunge 2 in cima allo stack
stack.append(3)     # Push: aggiunge 3 in cima allo stack

print(stack.pop())  # Pop: rimuove e restituisce 3, l'elemento in cima allo stack
print(stack.pop())  # Pop: rimuove e restituisce 2
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Stack e Queue: definizione e implementazione

Stack :: Operazioni di Base

- **Push:** Aggiunge un elemento in cima allo stack.
- **Pop:** Rimuove l'elemento in cima allo stack.
- **Top/Peak:** Restituisce l'elemento in cima allo stack senza rimuoverlo.

Come detto già, Python non ha una struttura dati stack dedicata, ma possiamo usare una lista per implementarne una.

```
stack = []

# Push
stack.append(1)
stack.append(2)
stack.append(3)

# Pop
print(stack.pop())  # Restituisce 3

# Top/Peak
print(stack[-1])  # Restituisce 2
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Stack e Queue: definizione e implementazione

Queue (Coda)

Una queue è una struttura dati di tipo FIFO (First In, First Out), dove il primo elemento aggiunto è il primo ad essere rimosso.

Operazioni di Base:

- **Enqueue:** Aggiunge un elemento alla fine della coda.
- **Dequeue:** Rimuove l'elemento all'inizio della coda.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Stack e Queue: definizione e implementazione

Queue (Coda)

Implementazione in Python:

Python fornisce il modulo **queue** che include una classe per implementare una coda. In alternativa, possiamo usare **collections.deque** che fornisce un'implementazione efficiente di una coda.

```
from collections import deque

queue = deque()

# Enqueue
queue.append(1)
queue.append(2)
queue.append(3)

# Dequeue
print(queue.popleft())  # Restituisce 1
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Stack e Queue: definizione e implementazione

Stack e Queue :: Considerazioni sull'Utilizzo

•**Stack:** È utile in situazioni in cui è necessario un accesso rapido all'elemento più recentemente aggiunto, come nel caso della valutazione delle espressioni e nel backtracking.
•**Queue:** È adatta per gestire un flusso di dati in ordine, come nell'elaborazione dei task o nella simulazione di code in ambienti reali (ad esempio, stampa di documenti, attività di server).

In sintesi, stack e queue sono strumenti fondamentali per la gestione dei dati in molti contesti algoritmici, e la loro scelta dipende dal tipo specifico di operazioni di inserimento e rimozione che devono essere eseguite sulla collezione di dati.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Linked Lists: introduzione e implementazione

Linked Lists

Le Linked Lists, o liste collegate, sono una struttura dati fondamentale in informatica. A differenza degli array o delle liste, che memorizzano gli elementi in posizioni contigue della memoria, le linked lists li memorizzano in nodi sparsi, ciascuno collegato al successivo tramite un riferimento o un puntatore.

Introduzione alle Linked Lists

1. **Nodi:** Ogni elemento di una linked list è contenuto in un "nodo". Ogni nodo ha due componenti: il dato e un riferimento (o puntatore) al nodo successivo nella lista.
2. **Testa della Lista:** La linked list è accessibile tramite la sua "testa", che è un riferimento al primo nodo della lista. Se la lista è vuota, la testa è **None**.
3. **Tipi di Linked Lists:** Ci sono diversi tipi, come la singolarmente collegata (singly linked list), dove ogni nodo punta solo al nodo successivo, e la doppiamente collegata (doubly linked list), dove ogni nodo ha riferimenti sia al nodo successivo sia a quello precedente.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Linked Lists: introduzione e implementazione

Implementazione di una Singly Linked List in Python

```python
class Nodo:
    def_init_(self, dato):
        self.dato = dato
        self.prossimo = None

class SinglyLinkedList:
    def__init__(self):
        self.testa = None

    def aggiungi(self, dato):
        nuovo_nodo = Nodo(dato)
        nuovo_nodo.prossimo = self.testa
        self.testa = nuovo_nodo

    def stampa_lista(self):
        corrente = self.testa
        while corrente:
            print(corrente.dato)
            corrente = corrente.prossimo

# Esempio d'uso
lista = SinglyLinkedList()
lista.aggiungi(3)
lista.aggiungi(2)
lista.aggiungi(1)
lista.stampa_lista()
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Algoritmi di ricerca: ricerca lineare e binaria

Ricerca

Gli algoritmi di ricerca sono fondamentali in informatica e vengono utilizzati per trovare un elemento in una struttura dati. Due degli algoritmi di ricerca più comuni sono la ricerca lineare e la ricerca binaria.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Algoritmi di ricerca: ricerca lineare e binaria

Ricerca Lineare (Sequential Search)

La ricerca lineare è il metodo più semplice. Scorre ogni elemento di una lista o array e controlla se corrisponde all'elemento desiderato.

Caratteristiche:

- **Semplicità:** Facile da implementare.
- **Efficienza su Liste Piccole:** Funziona bene su liste di piccole dimensioni.
- **Non richiede Ordinamento:** Può essere applicata a dati non ordinati.

Implementazione in Python:

```python
def ricerca_lineare(lista, elemento_da_trovare):
    for indice, elemento in enumerate(lista):
        if elemento == elemento_da_trovare:
            return indice
    return -1

lista = [3, 5, 2, 8, 6]
indice = ricerca_lineare(lista, 8)
print("Elemento trovato all'indice:", indice)
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Algoritmi di ricerca: ricerca lineare e binaria

Ricerca Binaria

La ricerca binaria è un algoritmo più efficiente che funziona su liste ordinate, dividendole ripetutamente a metà e scartando la metà in cui l'elemento desiderato non può trovarsi.

Caratteristiche:

- **Efficienza:** Molto più veloce della ricerca lineare su grandi liste ordinate.
- **Richiede una Lista Ordinata:** I dati devono essere ordinati per applicare la ricerca binaria.
- **Approccio Divide et Impera:** Divide l'insieme dei dati a metà in ogni passaggio.

Implementazione in Python:

```
def ricerca_binaria(lista,
    elemento_da_trovare): basso = 0
    alto = len(lista) - 1

    while basso <= alto:
        medio = (basso + alto) // 2
        guess = lista[medio]
        if guess == elemento_da_trovare:
            return medio
        if guess > elemento_da_trovare:
            alto = medio - 1
        else:
            basso = medio + 1
    return -1

lista_ordinata = [1, 3, 5, 7, 9]
indice = ricerca_binaria(lista_ordinata, 5)
print("Elemento trovato all'indice:", indice)
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Algoritmi di ricerca: ricerca lineare e binaria

Confronto

- **Ricerca Lineare:** Semplice, non richiede una lista ordinata, ma meno efficiente su grandi dataset.
- **Ricerca Binaria:** Richiede una lista ordinata, ma è molto più efficiente, specialmente su grandi dataset.

Mentre la ricerca lineare è un buon punto di partenza per piccole liste o liste non ordinate, la ricerca binaria è la scelta migliore per insiemi di dati più grandi che sono già stati ordinati, offrendo tempi di ricerca significativamente più rapidi.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Algoritmi di ordinamento: bubble sort, insertion sort e altri

Gli algoritmi di ordinamento sono fondamentali nell'informatica per organizzare i dati in un ordine specifico. Ci sono diversi algoritmi di ordinamento, ognuno con i suoi pro e contro in termini di complessità, efficienza e caso d'uso. Tra i più noti ci sono il Bubble Sort, l'Insertion Sort e altri come il QuickSort e il MergeSort.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Algoritmi di ordinamento: bubble sort, insertion sort e altri

Bubble Sort

Il Bubble Sort è uno degli algoritmi di ordinamento più semplici. Funziona ripetutamente scambiando elementi adiacenti se sono nell'ordine sbagliato.

Caratteristiche:

- **Semplicità:** Facile da capire e implementare.
- **Efficienza:** Molto inefficace per liste grandi; complessità temporale $O(n^2)$.

Implementazione in Python:

```
def bubble_sort(lista):
    n = len(lista)
    for i in range(n):
        for j in range(0, n-i-1):
            if lista[j] > lista[j+1]:
                lista[j], lista[j+1] = lista[j+1], lista[j]
    return lista
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Algoritmi di ordinamento: bubble sort, insertion sort e altri

Insertion Sort

L'Insertion Sort costruisce l'output ordinato un elemento per volta, inserendo ogni nuovo elemento nella posizione corretta nella parte già ordinata della lista.

Caratteristiche:

- **Semplice e Intuitivo:** Simile al modo in cui le persone ordinano le carte a mano.
- **Efficiente per Liste Piccole:** Buona performance su piccoli insiemi di dati.

Implementazione in Python:

```
def insertion_sort(lista):
    for i in range(1, len(lista)):
        chiave = lista[i]
        j = i-1
        while j >=0 and chiave < lista[j]:
            lista[j + 1] = lista[j]
            j -= 1
        lista[j + 1] = chiave
    return lista
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Algoritmi di ordinamento: bubble sort, insertion sort e altri

QuickSort

Il QuickSort è un algoritmo di ordinamento veloce e efficiente, basato sul principio del divide et impera.
Caratteristiche:

- **Efficiente:** Uno degli algoritmi di ordinamento più veloci per grandi insiemi di dati.
- **Complessità:** In media O(n log n), ma può degradare a $O(n^2)$ nel caso peggiore.

Implementazione in Python:

```
def quicksort(lista):
    if len(lista) <= 1:
        return lista
    pivot = lista[len(lista) // 2]
    sinistra = [x for x in lista if x < pivot]
    centro = [x for x in lista if x == pivot]
    destra = [x for x in lista if x > pivot]
    return quicksort(sinistra) + centro + quicksort(destra)
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Algoritmi di ordinamento: bubble sort, insertion sort e altri

MergeSort

Il MergeSort è un altro algoritmo di ordinamento efficiente basato sul principio del divide et impera.

Caratteristiche:

- **Efficiente e Stabile:** Complessità O(n log n) in tutti i casi.
- **Divide et Impera:** Divide l'insieme di dati in metà più piccole, le ordina e poi le unisce.

Implementazione in Python:

```
def mergesort(lista):
    if len(lista) > 1:
        centro = len(lista) // 2
        sinistra = lista[:centro]
        destra = lista[centro:]
        mergesort(sinistra)
        mergesort(destra)
        i = j = k = 0
        while i < len(sinistra) and j <
            len(destra): if sinistra[i] < destra[j]:
                lista[k] = sinistra[i]
                i += 1
            else:
                lista[k] = destra[j]
                j += 1
            k += 1
        while i < len(sinistra):
            lista[k] = sinistra[i]
            i += 1
            k += 1
        while j < len(destra):
            lista[k] = destra[j]
            j += 1
            k += 1
    return lista
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Algoritmi di ordinamento: bubble sort, insertion sort e altri

Conclusione

Ogni algoritmo di ordinamento ha i suoi vantaggi e svantaggi a seconda del contesto. Bubble Sort e Insertion Sort sono semplici ma meno efficienti per grandi insiemi di dati, mentre QuickSort e MergeSort sono molto più efficienti ma richiedono una comprensione più approfondita del principio di divide et impera. La scelta dell'algoritmo dipende dalla dimensione dei dati, dalla loro natura (già parzialmente ordinati o meno) e dai requisiti specifici di efficienza.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Analisi della complessità temporale e spaziale (Big O notation)

Complessità Temporale (Time Complexity)

La complessità temporale di un algoritmo misura il tempo necessario per eseguirlo rispetto alla dimensione dell'input. È spesso espressa in termini di notazione Big O, che descrive il limite superiore del tempo di esecuzione in funzione della dimensione dell'input.

Esempi Comuni di Complessità Temporale:

- **O(1) - Costante:** Il tempo di esecuzione non dipende dalla dimensione dell'input.
- **O(log n) - Logaritmica:** Tipico degli algoritmi che dividono l'input a metà in ogni passaggio (come la ricerca binaria).
- **O(n) - Lineare:** Il tempo di esecuzione cresce linearmente con l'incremento della dimensione dell'input.
- **O(n log n) - Log-lineare:** Comune in algoritmi efficienti di ordinamento (come QuickSort e MergeSort).
- **O(n^2) - Quadratica:** Spesso vista in algoritmi con doppio ciclo su dati (come il Bubble Sort).
- **O(2^n) - Esponenziale:** Cresce esponenzialmente con l'incremento dell'input. Comune in alcuni problemi di ottimizzazione o backtracking.
- **O(n!) - Fattoriale:** Cresce col fattoriale della dimensione dell'input. Tipico di algoritmi che generano tutte le permutazioni possibili.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Analisi della complessità temporale e spaziale (Big O notation)

Complessità Spaziale (Space Complexity)

La complessità spaziale misura la quantità di memoria richiesta da un algoritmo in funzione della dimensione dell'input.

Esempi Comuni di Complessità Spaziale:

- **O(1) - Costante:** La quantità di memoria utilizzata non cambia al variare della dimensione dell'input.
- **O(n) - Lineare:** La memoria necessaria cresce linearmente con l'incremento della dimensione dell'input.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Analisi della complessità temporale e spaziale (Big O notation)

Complessità Spaziale (Space Complexity)

La complessità spaziale misura la quantità di memoria richiesta da un algoritmo in funzione della dimensione dell'input.

Esempi Comuni di Complessità Spaziale:

- **O(1) - Costante:** La quantità di memoria utilizzata non cambia al variare della dimensione dell'input.
- **O(n) - Lineare:** La memoria necessaria cresce linearmente con l'incremento della dimensione dell'input.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Analisi della complessità temporale e spaziale (Big O notation)

Considerazioni sull'Uso della Notazione Big O

- **Stima Asintotica:** Big O fornisce una stima asintotica, focalizzandosi sul comportamento dell'algoritmo per grandi valori di n.
- **Peggiore Caso:** Solitamente, si considera il caso peggiore, a meno che non sia specificato diversamente.
- **Fattori Costanti e Termini di Ordine Inferiore:** Nell'analisi Big O, si ignorano i fattori costanti e i termini di ordine inferiore, poiché l'obiettivo è catturare la tendenza di crescita per grandi input.

L'analisi della complessità temporale e spaziale è cruciale per scegliere o progettare algoritmi adatti a un determinato problema, soprattutto quando si lavora con grandi quantità di dati o in sistemi con risorse limitate.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Strutture dati avanzate e utilizzo in Python

In Python, oltre alle strutture dati di base come liste, tuple, set e dizionari, esistono diverse strutture dati avanzate che possono essere utilizzate per risolvere problemi specifici in modo più efficiente. Queste strutture includono alberi, heap, grafici, code a priorità, e altre ancora. Di seguito, esploriamo alcune di queste strutture dati avanzate e il loro utilizzo in Python.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Strutture dati avanzate e utilizzo in Python

Alberi

Gli alberi sono strutture dati gerarchiche utilizzate in molti algoritmi e applicazioni. Un esempio comune è l'albero binario di ricerca, che permette inserimenti, ricerche e cancellazioni efficienti.

Utilizzo in Python:

Python non ha una libreria standard per gli alberi, ma è possibile implementarli manualmente o utilizzare librerie esterne come **bintrees** o **anytree**.

(ps: sugli alberi torneremo a parlare)

Heap (Monticoli)

Un heap è una struttura dati basata su albero utilizzata principalmente per implementare code a priorità. Gli heap preservano l'ordine in modo tale che l'elemento con il valore più alto o più basso sia sempre la radice dell'albero.

Utilizzo in Python:

Il modulo **heapq** in Python fornisce funzioni per implementare heap su liste, come **heappush** per aggiungere elementi e **heappop** per rimuovere l'elemento più piccolo.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Strutture dati avanzate e utilizzo in Python

Grafici

I grafici sono strutture dati che consistono di nodi (vertici) connessi da archi. Sono utili in molti contesti, come reti di computer, algoritmi di percorso, ecc.

Utilizzo in Python:

I grafici possono essere rappresentati tramite liste di adiacenza o matrici di adiacenza. Librerie come **networkx** offrono implementazioni potenti e flessibili per lavorare con grafici.

Code a Priorità

Una coda a priorità è una struttura dati che gestisce un insieme di elementi, ognuno con una "priorità". Elementi con priorità più alta vengono rimossi dalla coda per primi.

Utilizzo in Python:

Può essere implementata con un heap, usando il modulo **heapq**, o con classi apposite come **queue.PriorityQueue**.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Strutture dati avanzate e utilizzo in Python

Tabelle Hash (Hash Table)

Le tabelle hash implementano array associativi, una struttura che può mappare chiavi uniche a valori specifici.

Utilizzo in Python:

I dizionari Python sono un esempio di tabelle hash.

Strutture Dati per Stringhe

Per operazioni specifiche sulle stringhe, come il matching di pattern, sono utili strutture come trie o alberi di suffissi.

Utilizzo in Python:

Biblioteche come **pygtrie** forniscono implementazioni per trie.

Considerazioni sull'Uso

L'uso di queste strutture dati avanzate dipende fortemente dal problema specifico che si sta cercando di risolvere. Mentre le strutture di dati standard di Python sono sufficienti per molte applicazioni, la scelta di una struttura dati più avanzata può portare a un significativo aumento dell'efficienza, specialmente per applicazioni che richiedono operazioni complesse o gestiscono grandi quantità di dati.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Alberi e grafi: introduzione e concetti base

Alberi

(un piccolo approfondimento)

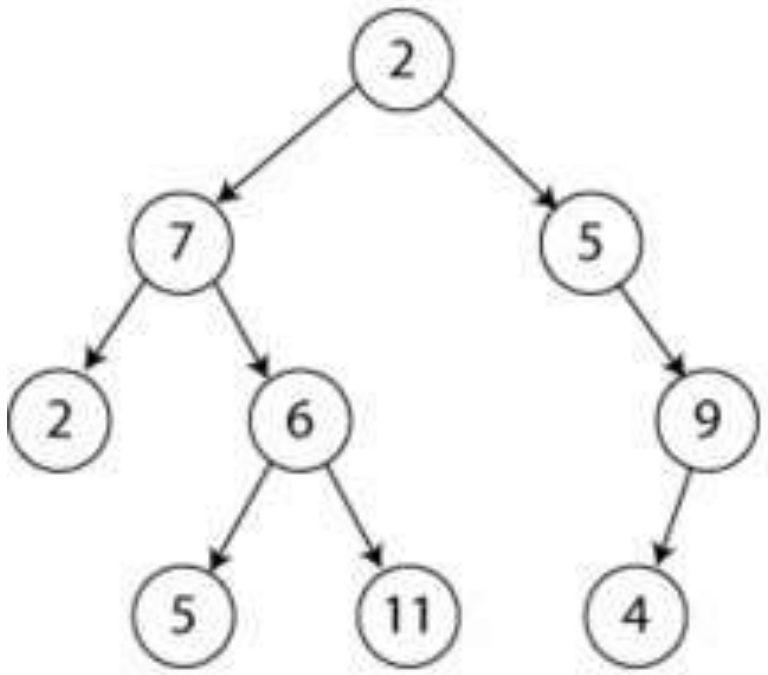
Gli alberi sono strutture dati gerarchiche che rappresentano relazioni di tipo genitore-figlio tra i nodi. Sono utilizzati in molte aree dell'informatica, come nei database, nell'elaborazione del linguaggio naturale e nelle strutture dati di base.

Concetti Base degli Alberi:

- **Nodo:** Un elemento dell'albero.
- **Radice:** Il nodo in cima all'albero, senza genitori.
- **Foglia:** Un nodo senza figli.
- **Profondità:** La distanza di un nodo dalla radice.
- **Altezza:** La distanza massima dalla radice a qualsiasi foglia.

Tipi di Alberi:

- **Albero Binario:** Ogni nodo ha al massimo due figli.
- **Albero Binario di Ricerca (BST):** Un albero binario in cui per ogni nodo, tutti gli elementi nel sottoalbero sinistro sono minori del nodo, e tutti nel sottoalbero destro sono maggiori.
- **Albero AVL, Albero Rosso-Nero:** Alberi binari di ricerca bilanciati, utilizzati per mantenere l'altezza dell'albero relativamente bassa.



Modulo 4

Strutture dati e algoritmi: un approccio pratico

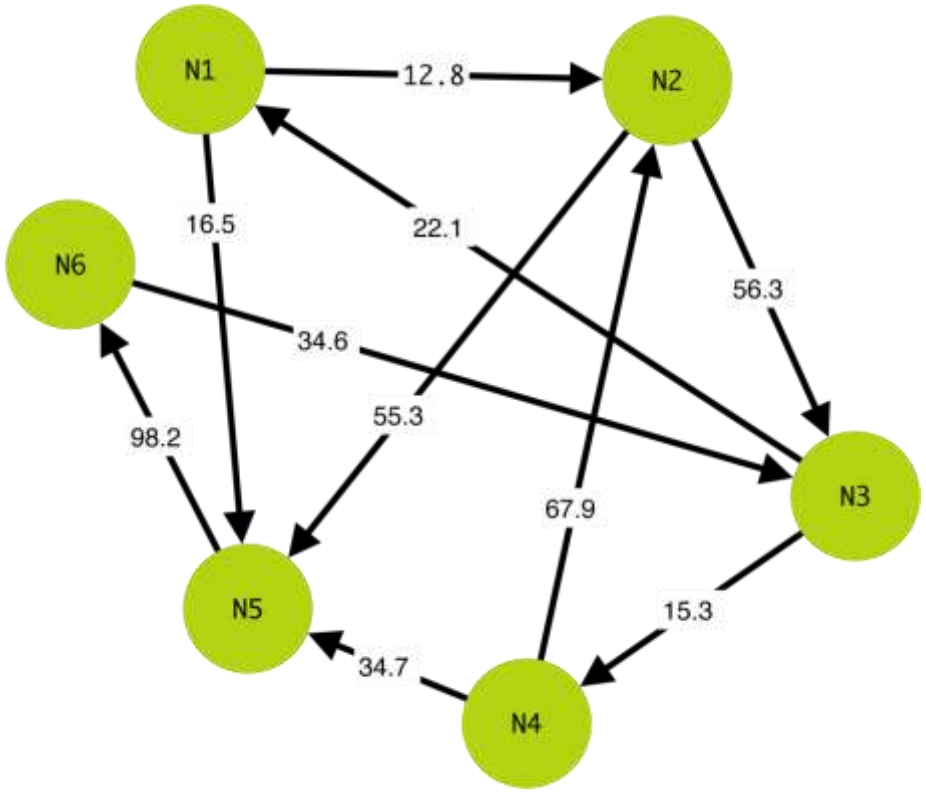
Alberi e grafi: introduzione e concetti base

Grafi

Un grafo è una struttura dati che consiste di un insieme di nodi (o vertici) e un insieme di archi che stabiliscono una relazione tra coppie di nodi.

Concetti Base dei Grafi:

- **Vertice (Nodo):** Un'unità di base del grafo che rappresenta un'entità.
- **Arco (Bordo):** Una connessione tra due vertici.
- **Grafo Diretto:** Un grafo in cui gli archi hanno una direzione.
- **Grafo Non Diretto:** Un grafo in cui gli archi non hanno direzione specifica.
- **Grafo Ponderato:** Un grafo in cui ogni arco ha un peso o costo associato.
- **Ciclo:** Un percorso che inizia e termina sullo stesso vertice.



Modulo 4

Strutture dati e algoritmi: un approccio pratico

Alberi e grafi: introduzione e concetti base

Utilizzo dei Grafi:

I grafi sono utilizzati per rappresentare reti di vario tipo, come reti sociali, reti di computer, mappe stradali, ecc. Possono essere utilizzati per risolvere problemi complessi come il percorso più breve, la pianificazione di rete, o l'analisi di reti sociali.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

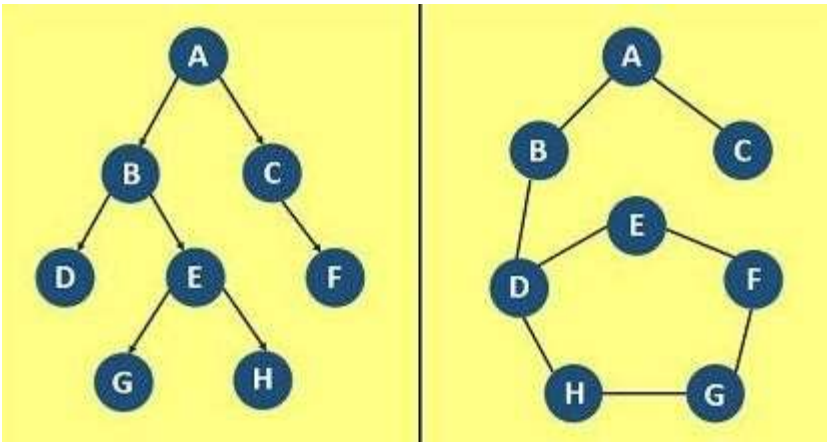
Alberi e grafi: introduzione e concetti base

Alberi vs Grafi

•**Struttura Gerarchica vs Rete:** Gli alberi sono una forma di grafo senza cicli (grafo aciclico) e con una struttura gerarchica, mentre i grafi possono avere cicli e una struttura più complessa.

•**Relazione Genitore-Figlio:** Negli alberi, ogni nodo (esclusa la radice) ha esattamente un genitore, mentre nei grafi, un nodo può avere più relazioni.

Sia gli alberi che i grafi sono strumenti potenti per modellare e risolvere una vasta gamma di problemi computazionali e sono ampiamente studiati nell'ambito dell'informatica teorica e applicata.



Modulo 4

Strutture dati e algoritmi: un approccio pratico

Introduzione alle hash tables

Concetti Base delle Hash Tables

1. **Chiave e Valore:** Una hash table memorizza dati in coppie chiave-valore. Ogni valore è associato a una chiave unica, che viene utilizzata per accedere velocemente al valore corrispondente.
2. **Funzione Hash:** Le hash tables utilizzano una funzione hash per convertire la chiave in un indice numerico, che determina dove il valore associato sarà memorizzato nella tabella. La funzione hash idealmente distribuisce le chiavi in modo uniforme attraverso la tabella.
3. **Collisioni:** A volte, due chiavi diverse possono produrre lo stesso indice hash, un evento noto come collisione. Le collisioni vengono gestite tramite varie tecniche, come il chaining (dove ogni cella della tabella hash mantiene una lista di tutti gli elementi il cui hash risulta in quell'indice) o l'indirizzamento aperto (dove si cerca un altro indice nella tabella).

Vantaggi delle Hash Tables

- **Efficienza di Ricerca/Inserimento/Cancellazione:** In condizioni ideali, queste operazioni possono essere svolte in tempo O(1), ovvero in tempo costante.
- **Ottimizzazione del Ricerca:** Ideali per casi in cui si prevede un grande numero di operazioni di ricerca.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Introduzione alle hash tables

Implementazione in Python

Python implementa le hash tables tramite i suoi dizionari (**dict**) e set. I dizionari Python sono collezioni di coppie chiave-valore, mentre i set sono collezioni di elementi unici.

Esempio con Dizionario Python:

```
# Creazione di un dizionario (hash table)
dizionario = {"nome": "Alice", "età": 30, "città": "Milano"}

# Accesso ai valori tramite chiave
print(dizionario["nome"])  # Output: Alice

# Aggiunta di una nuova coppia chiave-valore
dizionario["professione"] = "Ingegnere"

# Eliminazione di una coppia chiave-valore
del dizionario["età"]
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Introduzione alle hash tables

Considerazioni sull'Uso

•Le hash tables sono ottimali per situazioni che richiedono accessi rapidi ai dati, ma possono richiedere più memoria rispetto ad altre strutture dati come liste collegate.

•La scelta e l'implementazione della funzione hash sono cruciali per massimizzare l'efficienza e minimizzare le collisioni nella tabella.

In conclusione, le hash tables sono uno strumento potente e versatile in informatica, utilizzato per implementare strutture dati efficienti per lo stoccaggio e la ricerca di dati.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Design patterns avanzati: Singleton, Factory, Decorator, Observer, ecc.

I design pattern avanzati in programmazione sono metodi consolidati per risolvere problemi comuni di progettazione software. Sono utili per creare codice riutilizzabile e facile da manutenere. Ecco una panoramica di alcuni pattern avanzati come Singleton, Factory, Decorator e Observer.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Design patterns avanzati: Singleton, Factory, Decorator, Observer, ecc.

Singleton

Il pattern Singleton assicura che una classe abbia solo una singola istanza e fornisce un punto di accesso globale a tale istanza.

Caratteristiche:

•Controllo rigoroso sull'istanziazione.

•Spesso utilizzato per gestire risorse condivise, come connessioni al database.

Implementazione in Python:

```
class Singleton:
    _instance = None

    @classmethod
    def getInstance(cls):
        if cls._instance is None:
            cls._instance = cls()
        return cls._instance

# Utilizzo
singleton = Singleton.getInstance()
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Design patterns avanzati: Singleton, Factory, Decorator, Observer, ecc.

Factory

Il pattern Factory fornisce un'interfaccia per creare oggetti in una superclasse, ma permette alle sottoclassi di modificare il tipo di oggetti che verranno creati.

Caratteristiche:

•Promuove il loose coupling separando la creazione degli oggetti dalla loro implementazione.

•Utile quando un sistema deve essere indipendente da come i suoi prodotti vengono creati.

Implementazione in Python:

```python
class Automobile:
    def crea_auto(self):
        pass

class Ferrari(Automobile):
    def crea_auto(self):
        return "Ferrari"

class Lamborghini(Automobile):
    def crea_auto(self):
        return "Lamborghini"

def get_automobile(nome):
    if nome == 'Ferrari':
        return Ferrari()
    elif nome == 'Lamborghini':
        return Lamborghini()

# Utilizzo
auto = get_automobile("Ferrari")
print(auto.crea_auto())
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Design patterns avanzati: Singleton, Factory, Decorator, Observer, ecc.

Factory

Il pattern Factory fornisce un'interfaccia per creare oggetti in una superclasse, ma permette alle sottoclassi di modificare il tipo di oggetti che verranno creati.

Caratteristiche:

•Promuove il loose coupling separando la creazione degli oggetti dalla loro implementazione.

•Utile quando un sistema deve essere indipendente da come i suoi prodotti vengono creati.

Implementazione in Python:

```
class Automobile:
    def factory(tipo):
        if tipo == "Sportiva":
            return Sportiva()
        elif tipo == "Familiare":
            return Familiare()
        raise Exception("Tipo non
supportato")

    factory = staticmethod(factory)

class Sportiva(Automobile):
    pass

class Familiare(Automobile):
    pass

# Uso
auto = Automobile.factory("Sportiva")
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Design patterns avanzati: Singleton, Factory, Decorator, Observer, ecc.

Decorator

Aggiunge responsabilità aggiuntive a un oggetto in modo dinamico.

Utilizzo:

•Utilizzato per estendere o modificare il comportamento di oggetti runtime senza alterare la struttura delle classi esistenti.

•Utilizzato per aggiungere funzionalità agli oggetti senza modificarne il codice sorgente.

Implementazione in Python:

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        # Do something before
        result = func(*args, **kwargs)
        # Do something after
        return result
    return wrapper

@my_decorator
def my_function():
    pass
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Design patterns avanzati: Singleton, Factory, Decorator, Observer, ecc.

Observer

Definisce una dipendenza uno-a-molti tra oggetti in modo che quando un oggetto cambia stato, tutti i suoi dipendenti vengono notificati e aggiornati automaticamente.

Utilizzo:

•Utilizzato per costruire sistemi altamente interconnessi dove le azioni di un oggetto necessitano di risposte o cambiamenti in altri oggetti.

Implementazione in Python:

```
class Observer:
    def update(self, subject):
        pass

class Subject:
    def_init_(self):
        self._observers = []

    def attach(self, observer):
        self._observers.append(observer
)

    def notify(self):
        for observer in
self._observers:
            observer.update(self)
```

Questi design patterns forniscono soluzioni testate e affidabili per problemi ricorrenti nella progettazione del software e possono aiutare a rendere il codice più flessibile, riutilizzabile e manutenibile. La scelta del pattern giusto dipende dal problema specifico che si sta cercando di risolvere e dalla struttura del sistema software in cui si sta lavorando.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

I principi SOLID e la loro applicazione in Python

I principi SOLID sono un insieme di cinque principi di design dell'orientamento agli oggetti ideati da Robert C. Martin, spesso utilizzati per rendere il software più comprensibile, flessibile e manutenibile. Ecco una panoramica di ciascun principio e di come possono essere applicati in Python:

Modulo 4

Strutture dati e algoritmi: un approccio pratico

I principi SOLID e la loro applicazione in Python

Single Responsibility Principle (SRP)

Principio: Una classe dovrebbe avere una sola ragione per cambiare, il che significa che dovrebbe avere solo un lavoro o responsabilità.

Applicazione in Python: Progetta classi in modo che abbiano un singolo compito o responsabilità. Evita classi che gestiscono molteplici funzionalità.

```
class User:
    def__init__(self, name: str):
        self.name = name

class UserDB:
    def get_user(self, user_id: int):
        pass  # Restituisce l'utente dal
database
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

I principi SOLID e la loro applicazione in Python

Open/Closed Principle (OCP)

Principio: Le classi dovrebbero essere aperte per l'estensione, ma chiuse per la modifica. Ciò significa che dovresti essere in grado di aggiungere nuove funzionalità senza modificare il codice esistente.

Applicazione in Python: Usa l'ereditarietà e l'overriding (sovrascrittura) dei metodi per aggiungere funzionalità alle classi esistenti.

```
class Report:
    def generate(self):
        pass

class PDFReport(Report):
    def generate(self):
        # Implementazione specifica per PDF
        pass
```

In questo esempio, PDFReport estende Report e modifica il comportamento del metodo generate senza modificare la classe base Report.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

I principi SOLID e la loro applicazione in Python

Liskov Substitution Principle (LSP)

Definizione: Le classi derivate devono essere sostituibili con le loro classi base.

Applicazione in Python:

- Assicurarsi che le sottoclassi non modifichino il comportamento aspettato delle classi base.
- Usare astrazioni (come classi astratte o interfacce) in modo coerente.

```python
class Uccello:
    def vola(self):
        return "Questo uccello può volare"

class Passero(Uccello):
    def canta(self):
        return "Cip cip"

class Struzzo(Uccello):
    def vola(self):
        return "Questo uccello non può volare"

# Funzione che lavora con oggetti
Uccello def fai_volare(uccello):
    print(uccello.vola())

# Istanze delle classi
passero = Passero()
struzzo = Struzzo()

# Test delle funzioni
fai_volare(passero)  # Output: Questo uccello può volare
fai_volare(struzzo)  # Output: Questo uccello non può volare
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

I principi SOLID e la loro applicazione in Python

```python
class Uccello:
    def vola(self):
        return "Questo uccello può volare"

class Passero(Uccello):
    def canta(self):
        return "Cip cip"

class Struzzo(Uccello):
    def vola(self):
        return "Questo uccello non può volare"

# Funzione che lavora con oggetti
Uccello def fai_volare(uccello):
    print(uccello.vola())

# Istanze delle classi
passero = Passero()
struzzo = Struzzo()

# Test delle funzioni
fai_volare(passero)  # Output: Questo uccello può volare
fai_volare(struzzo)  # Output: Questo uccello non può
volare
```

Liskov Substitution Principle (LSP)

:: Spiegazione

In questo esempio:

- Abbiamo una classe base **Uccello** con un metodo **vola**.
- **Passero** è una sottoclasse di **Uccello** e non modifica il comportamento del metodo **vola**.
- **Struzzo** è un'altra sottoclasse di **Uccello**, ma modifica il comportamento di **vola** per riflettere che gli struzzi non possono volare.

La funzione **fai_volare** accetta un oggetto **Uccello** e chiama il suo metodo **vola**. Secondo il LSP, possiamo passare un oggetto **Passero** o **Struzzo** alla funzione **fai_volare**, e il programma funziona correttamente perché entrambe le classi seguono il contratto definito dalla loro classe base **Uccello**.

Questo esempio dimostra il LSP in azione: le sottoclassi **Passero** e **Struzzo** possono essere usate al posto della classe **Uccello** senza causare errori o comportamenti inattesi nel programma, nonostante modifichino o estendano il comportamento della classe base.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

I principi SOLID e la loro applicazione in Python

Interface Segregation Principle (ISP)

Definizione: nessuna classe dovrebbe essere costretta a dipendere da metodi che non utilizza. Questo principio promuove l'utilizzo di interfacce più specifiche invece di una singola interfaccia generale, riducendo il rischio di implementare metodi non necessari nelle classi.

Applicazione in Python:

•Creare interfacce specifiche piuttosto che un'unica interfaccia generale.

•Utilizzare interfacce più piccole e focalizzate (in Python, spesso raggiunto tramite l'uso di mix-ins).

```python
from abc import ABC, abstractmethod

class IStampante(ABC):
    @abstractmethod
    def stampa(self, document):
        pass

class IScanner(ABC):
    @abstractmethod
    def scansiona(self, document):
        pass

class IFax(ABC):
    @abstractmethod
    def invia_fax(self, document):
        pass
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

I principi SOLID e la loro applicazione in Python

Interface Segregation Principle (ISP) :: Implementazione delle Classi Specifiche

Ora, possiamo creare classi specifiche che implementano solo le interfacce di cui hanno bisogno.

```
class StampanteMultifunzione(IStampante, IScanner,
    IFax): def stampa(self, document):
        print(f"Stampa: {document}")

    def scansiona(self, document):
        print(f"Scansione: {document}")

    def invia_fax(self, document):
        print(f"Invia fax: {document}")

class StampanteSemplice(IStampante):
    def stampa(self, document):
        print(f"Stampa: {document}")

class ScannerSemplice(IScanner):
    def scansiona(self, document):
        print(f"Scansione: {document}")
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

I principi SOLID e la loro applicazione in Python

Interface Segregation Principle (ISP) :: Implementazione delle Classi Specifiche

Ora, possiamo creare classi specifiche che implementano solo le interfacce di cui hanno bisogno.

```
class StampanteMultifunzione(IStampante, IScanner,
    IFax): def stampa(self, document):
        print(f"Stampa: {document}")

    def scansiona(self, document):
        print(f"Scansione: {document}")

    def invia_fax(self, document):
        print(f"Invia fax: {document}")

class StampanteSemplice(IStampante):
    def stampa(self, document):
        print(f"Stampa: {document}")

class ScannerSemplice(IScanner):
    def scansiona(self, document):
        print(f"Scansione: {document}")
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

I principi SOLID e la loro applicazione in Python

Interface Segregation Principle (ISP) :: Utilizzo delle Classi

Infine, possiamo creare e utilizzare istanze di queste classi.

```
# Creazione delle istanze
stampante_multifunzione =
StampanteMultifunzione() stampante_semplice =
StampanteSemplice()  scanner_semplice =
ScannerSemplice()

# Utilizzo delle funzionalità
stampante_multifunzione.stampa("Documento
PDF") scanner_semplice.scansiona("Foto")
# stampante_semplice.scansiona("Documento")  # Errore:
questa classe non ha il metodo scansiona
```

Spiegazione

In questo esempio, **StampanteMultifunzione** implementa tre interfacce (**IStampante**, **IScanner**, **IFax**), mentre **StampanteSemplice** e **ScannerSemplice** implementano rispettivamente solo **IStampante** e **IScanner**. Questo approccio aderisce all'ISP, poiché le classi non sono costrette a implementare metodi che non utilizzeranno, evitando così la necessità di dipendenze e implementazioni non necessarie.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

I principi SOLID e la loro applicazione in Python

Dependency Inversion Principle (DIP)

Per questo particolare caso rimando a dopo aver parlato della Dependency Injection

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Dependency Injection in Python

Dependency Injection (DI)

La Dependency Injection (DI) è un principio di progettazione software utilizzato per migliorare la modularità e la flessibilità del codice, facilitando allo stesso tempo il testing. In Python, la DI può essere implementata in diversi modi, senza necessariamente utilizzare framework complessi come accade in altri linguaggi.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Dependency Injection in Python

Cos'è la Dependency Injection?

La DI è un pattern che permette di rimuovere le dipendenze hardcoded all'interno di una classe, fornendo invece queste dipendenze dall'esterno, spesso tramite il costruttore, un setter o un metodo di fabbrica. Questo rende le classi meno accoppiate e più facili da testare, poiché le dipendenze possono essere facilmente sostituite, ad esempio con mock durante il testing.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Dependency Injection in Python

Esempio di DI in Python

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Dependency Injection in Python

Senza Dependency Injection

```
class ServizioLog:
    def log(self, messaggio):
        print(f"Log: {messaggio}")

class Applicazione:
    def __init__(self):
        self.servizio_log = ServizioLog()

    def run(self):
        self.servizio_log.log("Applicazione in esecuzione")
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Dependency Injection in Python

Con Dependency Injection

```
class Applicazione:
    def_init_(self, servizio_log):
        self.servizio_log = servizio_log

    def run(self):
        self.servizio_log.log("Applicazione in esecuzione")

# Creazione delle istanze
servizio_log  =  ServizioLog()
app = Applicazione(servizio_log)

# Esecuzione dell'applicazione
app.run()
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Dependency Injection in Python

Nel secondo esempio, **Applicazione** riceve il **ServizioLog** come dipendenza esterna, rendendo la classe **Applicazione** meno accoppiata e più flessibile. Per esempio, per il testing, possiamo sostituire **ServizioLog** con un mock o un stub che implementa la stessa interfaccia.

Vantaggi della Dependency Injection

1. **Riduzione dell'Accoppiamento:** Le classi non creano più direttamente le proprie dipendenze, il che le rende più flessibili e riutilizzabili.
2. **Migliore Supporto per il Testing:** Le dipendenze possono essere sostituite con versioni mock o stub per facilitare il testing.
3. **Maggiore Chiarezza e Manutenibilità:** Le dipendenze di una classe diventano più evidenti.

Considerazioni

Mentre la DI è un potente strumento di progettazione, è importante usarla in modo appropriato. Un eccessivo uso di DI può portare a complessità aggiuntiva, specialmente in progetti più piccoli o meno complessi. In Python, grazie alla sua natura dinamica, spesso è possibile utilizzare la DI in modo semplice ed elegante senza la necessità di strumenti aggiuntivi.

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Dependency Injection in Python

Dependency Inversion Principle (DIP)

Definizione: Le classi ad alto livello non dovrebbero dipendere da classi a basso livello. Entrambe dovrebbero dipendere da astrazioni.

Applicazione in Python:

- Utilizzare l'iniezione di dipendenza per disaccoppiare le classi di alto livello dalle classi di basso livello.
- Definire le dipendenze come astrazioni (ad esempio, usando classi astratte o interfacce), piuttosto che come implementazioni concrete.

```python
from abc import ABC, abstractmethod
# SRP & OCP: Classe con responsabilità singola e aperta all'estensione
class ReportGenerator(ABC):
    @abstractmethod
    def generate_report(self, data):
        pass
# LSP: Sottoclasse che può sostituire la classe base
class PDFReportGenerator(ReportGenerator):
    def generate_report(self, data):
        # Implementazione specifica per PDF
        pass
# ISP: Utilizzo di interfacce specifiche piuttosto che generali
class DataFetcher:
    def fetch_data(self):
        # Logica per recuperare i dati
        pass
# DIP: Utilizzo dell'iniezione di dipendenza
class ReportService:
    def_init_(self, report_generator, data_fetcher):
        self.report_generator = report_generator
        self.data_fetcher = data_fetcher

    def create_report(self):
        data = self.data_fetcher.fetch_data()
        return self.report_generator.generate_report(data)
# Creazione di istanze
pdf_generator = PDFReportGenerator()
data_fetcher = DataFetcher()
report_service = ReportService(pdf_generator, data_fetcher)
# Generazione del report
report_service.create_report()
```

Modulo 4

Strutture dati e algoritmi: un approccio pratico

Dependency Injection in Python

In questo esempio, ogni principio SOLID è applicato per creare un sistema di generazione di report flessibile e manutenibile. Utilizzando questi principi, il codice diventa più chiaro, più facile da estendere e da mantenere, specialmente in progetti software di grandi dimensioni.